

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
BỘ MÔN KHOA HỌC MÁY TÍNH

THIẾT KẾ VÀ ĐÁNH GIÁ THUẬT TOÁN

Chương 3

Phương pháp chia để trị (*Divide and conquer*)

Phạm Hồng Phong
phongph@huce.edu.vn

Nội dung

1. **Giới thiệu**
2. **Bài toán tìm kiếm nhị phân**
3. **Bài toán Min – Max**
4. **Bài toán Cặp điểm gần nhất**

§ 1. Giới thiệu

- **Ý tưởng giải thuật chia để trị:**

Là phương pháp thiết kế thuật toán dựa trên 2 thao tác chính:

- Chia (*divide*): phân rã bài toán ban đầu thành các bài toán con ***có kích thước nhỏ hơn*** và ***có cùng cách giải*** với bài toán ban đầu.
- Trị (*conquer*): giải từng bài toán con (theo cách tương tự bài toán đầu - đệ quy) rồi tổng hợp các lời giải để nhận kết quả của bài toán ban đầu.

Việc “Phân rã”: thực hiện trên miền dữ liệu đến lúc miền dữ liệu đủ nhỏ để bài toán có thể giải xác định (trường hợp suy biến trong đệ quy)

§ 1. Giới thiệu

- **Mô hình**

Xét bài toán P trên miền dữ liệu R.

Gọi $D\&C(R)$ là thuật giải P trên miền dữ liệu R.

Nếu R có thể phân rã thành n miền con ($R = R_1 \cup R_2 \cup \dots \cup R_k$)

Với R_0 là miền đủ nhỏ để $D\&C(R_0)$ có lời giải thì lược đồ giải thuật chia để trị:

$D\&C(R) \equiv$

if ($R=R_0$)

 Giải $D\&C(R_0)$;

else

 Chia miền R thành $R_1, R_2, \dots, R_k$

for ($i=1\dots k$)

$D\&C(R_i)$

 Tổng hợp để nhận lời giải.

End.

§ 1. Giới thiệu

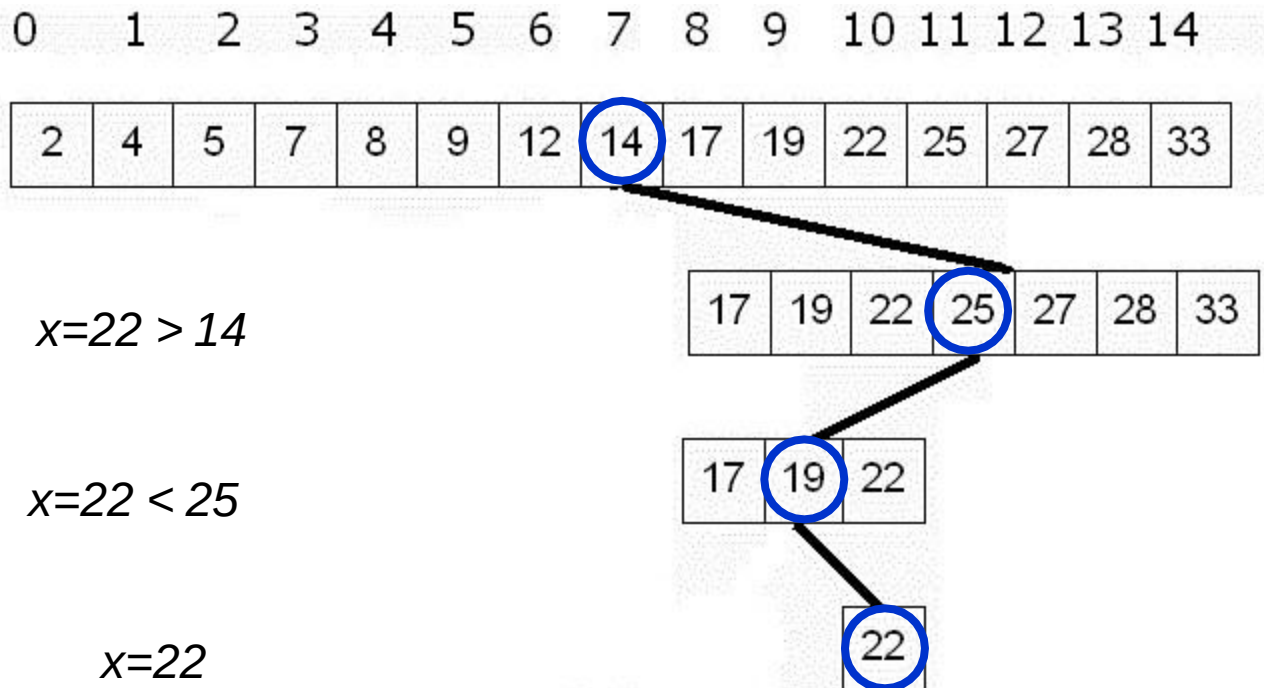
- **Tính đúng của giải thuật chia để trị**
 - Chia để trị sử dụng kỹ thuật đệ quy, thông thường là đệ quy nhiều nhánh
 - Tính đúng của thuật toán D&C có thể được chứng minh như đối với giải thuật đệ quy (sử dụng quy nạp).
 - Quy nạp
 - Cơ sở quy nạp: Giải thuật đúng trong trường hợp suy biến R_0
 - Giả thiết quy nạp: Giả sử giải thuật đúng với mọi miền dữ liệu có kích thước $\leq k$
 - Tổng quát: Chứng minh giải thuật đúng với miền dữ liệu có kích thước bằng $k + 1$

§ 2. Bài toán tìm kiếm nhị phân

- **Bài toán: Tìm kiếm nhị phân trên mảng được sắp**
 - Cho mảng n phần tử được sắp theo thứ tự (tăng dần) và một giá trị x bất kỳ. Kiểm tra xem phần tử x có trong dãy không? Nếu có, trả về vị trí của x
 - Ý tưởng: so sánh giá trị x với phần tử giữa của dãy tìm kiếm. Dựa vào giá trị này sẽ quyết định giới hạn tìm kiếm ở bước kế tiếp là nửa trước hay nửa sau dãy.

§ 2. Bài toán tìm kiếm nhị phân

Tìm $x = 22$ trong dãy được sắp:



§ 2. Bài toán tìm kiếm nhị phân

- **Lược đồ thuật toán**

```
BinarySearch(a,x, L,R):int =  
    //Tìm kiếm phần tử x trên dãy a từ vị trí L đến R  
    if (L=R) return (x=a[L] ? L : -1)  
    else  
        M = (L+R)/2;  
        if (x = a[M]) return (M);  
        else  
            if (x<a[M]) BinarySearch(a,x,L,M-1)  
            else BinarySearch(a,x,M+1,R)  
        endif;  
    endif;  
End.
```


§ 2. Bài toán tìm kiếm nhị phân

- **Lược đồ chương trình** có lời gọi BinarySearch

```
main() ≡  
    Input (a,n);  
    Input x;  
    result = BinarySearch(a,x,0,n-1);  
    Output (result);  
End.
```

§ 2. Bài toán tìm kiếm nhị phân

- **Chứng minh tính đúng của thuật toán**

Chứng minh quy nạp theo n (số phần tử của dãy)

- Cơ sở quy nạp: $n = 1$

Vì $L = R$ nên thuật toán trả về $(x=a[L] ? L : -1)$

=> Thuật toán đúng với $n = 1$

```
BinarySearch(a, x, L, R) : int =  
    //Tìm kiếm phần tử x trên dãy a từ vị trí L đến R  
    if (L=R) return (x=a[L] ? L : -1)  
    else  
        M = (L+R)/2;  
        if (x = a[M]) return (M);  
        else  
            if (x < aM) BinarySearch(a, x, L, M-1)  
            else BinarySearch(a, x, M+1, R)  
        endif;  
    endif;  
End.
```

§ 2. Bài toán tìm kiếm nhị phân

- Giả thiết quy nạp: Giả sử thuật toán đúng với mọi dãy có độ dài $n \leq k$
- Tổng quát: Chứng minh thuật toán đúng với mọi dãy có độ dài $n = k+1$
 - Nếu $x=a[M]$ thì kết quả trả về là M : thuật toán đúng
 - Nếu $x < a[M]$ thuật toán gọi tới $\text{BinarySearch}(a,x,L,M-1)$

Theo giả thiết quy nạp thì $\text{BinarySearch}(a,x,L,M-1)$ đúng

TH1: x có mặt trong dãy $\Rightarrow x$ thuộc $a[L]..A[M-1]$

$\text{BinarySearch}(a,x,L,M-1)$ trả về vị trí của x

$\Rightarrow$ thuật toán đúng

TH2: x không có mặt trong dãy $\Rightarrow x$ không thuộc $a[L]..A[M-1]$

$\text{BinarySearch}(a,x,L,M-1)$ trả về -1

$\Rightarrow$ thuật toán đúng

- Nếu $x > a[M]$ tương tự

§ 2. Bài toán tìm kiếm nhị phân

- **Độ phức tạp: Sử dụng định lý chính**

$$T(n) = \begin{cases} O(1) & \text{khi } n = 1 \\ T(n/2) + O(1) & \text{khi } n > 1 \end{cases}$$

Ta có $a = 1; b = 2; d = 0$

$$\Rightarrow a = b^d$$

$$\Rightarrow T(n) = O(n^d \log n) = O(\log n)$$

§ 3. Bài toán min - max

- **Bài toán: Tìm phần tử lớn nhất, nhỏ nhất của một dãy**
 - Bài toán: Cho mảng n phần tử, tìm giá trị lớn nhất và nhỏ nhất các phần tử trong dãy.
 - Phân tích:
 - Phương pháp truyền thống: Dùng hai vòng lặp độc lập tìm max , min .
 - Độ phức tạp thuật toán $O(n)$
 - Số phép so sánh $2(n-1)$
 - Phương pháp chia để trị:
 - Chia: Chia dãy thành 2 phần
 - Trị: Tìm giá trị max , min trong mỗi phần
 - Kết hợp: So sánh giá trị min , max đã tìm được

§ 3. Bài toán min - max

- **Lược đồ thuật toán**

MinMax(a, L, R) : (int, int) $\equiv$

```
//Tìm giá trị min,max dãy a từ vị trí L đến R
if (R-L<=1) {
    return(min(aL, aR), max(aL, aR));
}
else {
    (min1, max1) = MinMax(a, L, (L+R)/2);
    (min2, max2) = MinMax(a, (L+R)/2+1, R);
    return(min(min1, min2), max(max1, max2));
}
```

End.

Lời gọi ban đầu với dãy n phần tử: MinMax(a, 0, n-1)

§ 3. Bài toán min - max

- **Tính đúng của thuật toán**

Chứng minh quy nạp theo n là số phần tử của dãy

- Cơ sở: Với $n \leq 1$, thuật toán trả về $\min(a_L, a_R)$, $\max(a_L, a_R)$

=> Thuật toán đúng

- Giả thiết quy nạp: Thuật toán đúng với mọi $n \leq k$

§ 3. Bài toán min - max

- **Tính đúng của thuật toán**

- Tổng quát: Chứng minh thuật toán đúng với $n = k+1$

```
(min1, max1) = MinMax(a, L, (L+R)/2);  
(min2, max2) = MinMax(a, (L+R)/2+1, R);  
return(min(min1, min2), max(max1, max2));
```

Theo giả thiết quy nạp:

$\text{min1} = \min(a[L], \dots, a[(L+R)/2])$

$\text{min2} = \min(a[(L+R)/2+1], \dots, a[R])$

$\Rightarrow \min(\text{min1}, \text{min2}) = \min(a[L], \dots, a[R])$

Tương tự: $\text{max}(\text{max1}, \text{max2}) = \max(a[L], \dots, a[R])$

$\Rightarrow$ Thuật toán đúng

§ 3. Bài toán min - max

- **Tìm phần tử lớn nhất, nhỏ nhất của dãy**

- Độ phức tạp thuật toán

$$T(n) = \begin{cases} O(1) & \text{khi } n \leq 2 \\ 2T(n/2) + O(1) & \text{khi } n > 2 \end{cases}$$

$$\Rightarrow T(n) = O(n)$$

$T(n)$ không khác với phương pháp truyền thống (sử dụng 2 vòng lặp)!!!

§ 3. Bài toán min - max

Phân tích thêm: Đếm số phép so sánh $C(n)$: (Giả sử $n=2^m$)

$$C(n) = C(2^m) = 2C(2^{m-1}) + 2$$

$$= 2(2C(2^{m-2}) + 2) + 2 = 2^2C(2^{m-2}) + 2^2 + 2$$

$$= \dots$$

$$= 2^{m-1}C(2) + 2^{m-1} + \dots + 2^2 + 2 = 2^{m-1} + (2^{m-1} + \dots + 2^2 + 2) \text{ (do } C(2)=1)$$

$$= n/2 + 2(2^{m-1} - 1) = 3(n/2) - 2$$

$\Rightarrow$ Số phép so sánh bằng khoảng 3/4 so với phương pháp truyền thống

```
(min1,max1) = MinMax(a,L,(L+R)/2);
```

```
(min2,max2) = MinMax(a,(L+R)/2+1,R);
```

```
return(min(min1,min2),max(max1,max2));
```

§ 4. Bài toán tìm cặp điểm gần nhất

- **Bài toán: Tìm kiếm cặp điểm gần nhất (closest-pair problem)**
 - Cho một tập n điểm trong mặt phẳng, tìm cặp điểm gần nhau nhất (theo khoảng cách Euclid)

$$P = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

Tìm cặp điểm p_i, p_j sao cho $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ nhỏ nhất

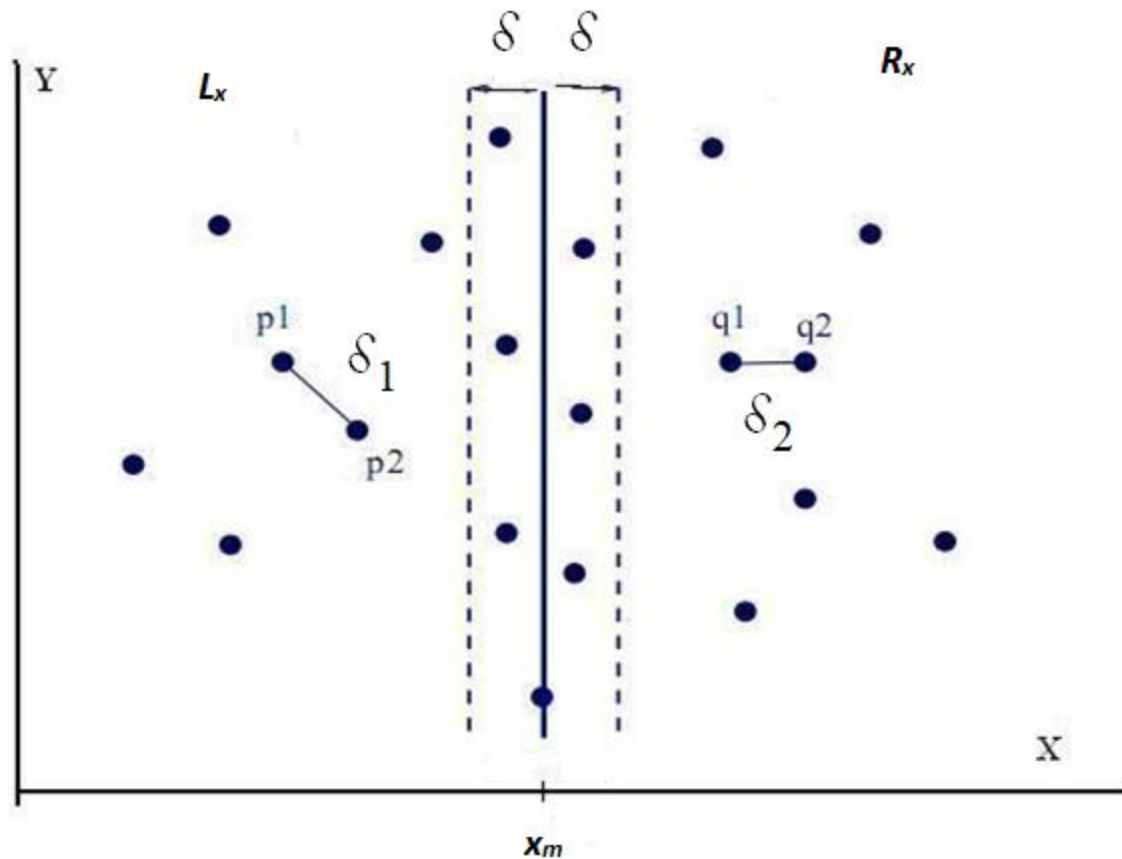
- Nếu dùng phương pháp vét cạn: So sánh khoảng cách mọi cặp điểm
=> độ phức tạp là $O(n^2)$
- Tìm giải thuật tốt hơn?

§ 4. Bài toán tìm cặp điểm gần nhất

- **Phân tích giải pháp chia để trị**

- Chia: Phân chia tập n điểm P thành 2 tập con L_x, R_x (theo giá trị tăng dần của hoành độ)
- Trị: Tìm cặp điểm gần nhất trong mỗi tập L_x, R_x . Giả sử khoảng cách giữa hai điểm gần nhau nhất của L_x là δ_1 , của R_x là δ_2 .
- Kết hợp: So sánh và lấy ra cặp điểm gần hơn $\delta = \min(\delta_1, \delta_2)$
- Chú ý:
 - Có thể có cặp (1 điểm thuộc L_x , 1 điểm thuộc R_x) có khoảng cách ngắn hơn
 - Cần xem xét thêm các điểm có hoành độ thuộc khoảng $(x_m - \delta, x_m + \delta)$

§ 4. Bài toán tìm cặp điểm gần nhất



§ 4. Bài toán tìm cặp điểm gần nhất

- **Lược đồ thuật toán**

CLOSESTPAIR(P)

▷ P is a set of at least two points on the plane, given by their x - and y -coordinates

P_x := the list of points in P sorted by x -coordinate $O(n \cdot \log n)$

P_y := the list of points in P sorted by y -coordinate $O(n \cdot \log n)$

return RCP(P_x, P_y)

RCP(P_x, P_y)

▷ P_x and P_y are lists of the same set of (at least two) points sorted by x - and y -coordinate, respectively

```
1  if  $|P_x| \leq 3$  then
2      calculate all pairwise distances and return the closest pair           $O(1)$ 
3  else
4       $L_x$  := first half of  $P_x$ ;  $R_x$  := second half of  $P_x$                  $O(n)$ 
5       $m$  := (max  $x$ -coordinate in  $L_x$  + min  $x$ -coordinate in  $R_x$ )/2         $O(1)$ 
6       $L_y$  := sublist of  $P_y$  of points in  $L_x$ ;  $R_y$  := sublist of  $P_y$  of points in  $R_x$      $O(n)$ 
7       $(p_L, q_L)$  := RCP( $L_x, L_y$ );  $(p_R, q_R)$  := RCP( $R_x, R_y$ )           $2 * T_{\text{RCP}}(n/2)$ 
8       $\delta$  := min( $d(p_L, q_L), d(p_R, q_R)$ )                                 $O(1)$ 
9      if  $d(p_L, q_L) = \delta$  then  $(p^*, q^*) := (p_L, q_L)$  else  $(p^*, q^*) := (p_R, q_R)$      $O(1)$ 
10      $B$  := sublist of  $P_y$  of points whose  $x$ -coordinates are within  $\delta$  of  $m$      $O(n)$ 
11     for each  $p \in B$  in order of appearance on  $B$  do
12         for each of the (up to) next seven points  $q$  after  $p$  on  $B$  do     $O(n)$ 
13             if  $d(p, q) < d(p^*, q^*)$  then  $(p^*, q^*) := (p, q)$ 
14     return  $(p^*, q^*)$                                                      $O(1)$ 
```

§ 4. Bài toán tìm cặp điểm gần nhất

- **Độ phức tạp thuật toán ClosestPair**

- Sắp xếp tập: $O(n \log n)$
- RCP:

$$T_{RCP}(n) = \begin{cases} O(1) & \text{khi } n \leq 3 \\ 2T_{RCP}\left(\frac{n}{2}\right) + O(n) & \text{khi } n > 3 \end{cases}$$

$$\Rightarrow T_{RCP}(n) = O(n \log n)$$

$$\Rightarrow T_{ClosestPair}(n) = O(n \log n)$$

Bài tập

Tìm hiểu một số thuật toán chia để trị

- Trình bày thuật toán sắp xếp trộn, so sánh với các thuật toán sắp xếp có độ phức tạp $O(n^2)$.
- Trình bày thuật toán tìm kiếm nhị phân, so sánh với tìm kiếm bằng cách duyệt từ đầu đến cuối mảng.
- Trình bày thuật toán tìm max-min bằng cách chia để trị, so sánh với tìm max-min bằng cách duyệt từ đầu đến cuối mảng.

Bài tập

Tìm hiểu một số thuật toán chia để trị

- Trình bày thuật toán tìm dãy con liên tiếp có tổng lớn nhất, so sánh với tìm dãy con liên tiếp có tổng lớn nhất bằng cách thực hiện hai vòng lặp lồng nhau.
- Trình bày thuật toán tìm cặp điểm gần nhất bằng cách chia để trị, so sánh với tìm cặp điểm gần nhất bằng cách duyệt từng cặp.